

Revisiting *Toy Models of Superposition*

Anonymous authors

Paper under double-blind review

Abstract

Mechanistic interpretability seeks to illuminate the internal workings of trained models by mapping latent computations to human-understandable structures. A key concept in this line of research is superposition, where multiple features are encoded within a compressed representational space. In this paper, we replicate and extend the foundational experiments from *Toy Models of Superposition* (Elhage et al., 2022b). First, we confirm their core findings through independent implementation. Then, we extend the approach to more complex architectures, including GPT-2-style transformers and encoder-decoder translation models. We further propose a unified framework connecting *representational superposition*—packing features into limited dimensions—with *reasoning superposition*—maintaining multiple reasoning traces in continuous latent space, as studied in recent work on chain-of-continuous-thought. Our results demonstrate that superposition is not merely an artifact of simplified experimental settings but a fundamental property of neural networks that persists across model architectures and scales to practical language models. These findings have important implications for AI safety and alignment, as understanding superposition enables techniques for steering model behavior and avoiding harmful outputs.

1 Introduction

Deep neural networks have achieved remarkable performance across diverse tasks, from image classification to natural language processing. However, these systems remain largely “black boxes,” making it difficult to understand how their outputs are derived. This lack of interpretability becomes particularly problematic when models produce unexpected or harmful behaviors, such as hallucinations or biased outputs.

Mechanistic interpretability has emerged as a promising approach to address this challenge. Originally developed for analyzing convolutional neural networks and image classifiers (Olah et al., 2017; Cammarata et al., 2020), the field aims to reverse-engineer models into human-understandable concepts. With the rise of large language models, researchers have increasingly focused on interpreting transformer architectures, though significant challenges remain in understanding these large-scale systems.

A central concept in mechanistic interpretability is *superposition*—the phenomenon where neural networks represent more features than they have dimensions by encoding multiple features in overlapping directions. Understanding superposition is crucial for AI safety: recent work has shown that identifying superposed features enables steering models away from harmful behaviors (Templeton et al., 2024). If we cannot disentangle the features a model has learned, we cannot reliably predict or control its behavior.

In this work, we replicate the foundational results from *Toy Models of Superposition* (Elhage et al., 2022b) and extend them to more complex architectures. While the original paper included code, our independent reimplementations serve to validate these influential findings, which have accumulated over 3,700 citations and shaped subsequent research in mechanistic interpretability. More importantly, we demonstrate that superposition—initially observed in simple autoencoders—persists in practical architectures including GPT-2-style transformers and encoder-decoder translation models.

Contributions. We make three main contributions. First, we provide an independent replication of the original superposition results in toy autoencoder models, validating the core findings of Elhage et al. (2022b)

(Section ??). Second, we extend these results to more complex architectures, demonstrating that superposition emerges in GPT-2-style transformers trained on synthetic sparse features, in MarianMT translation models with constrained bottleneck layers, and in continuous chain-of-thought models with iterative reasoning (Section ??). Third, motivated by the consistent emergence of superposition across these diverse settings, we propose a unified framework connecting *representational superposition*—packing features into limited dimensions—with *reasoning superposition*—maintaining multiple reasoning traces in continuous latent space (Hao et al., 2024; Zhu et al., 2025b)—and validate this framework empirically by comparing single-step and iterative models under controlled compression (Sections 6–7).

All code and results are available at https://github.com/mmjerge/superposition_replication.

2 Background

2.1 Theoretical Foundations

The study of feature representation in neural networks builds upon theoretical work in compressed sensing (Donoho, 2006) and distributed representations (Thorpe, 1989). Compressed sensing demonstrates how sparse signals can be recovered from lower-dimensional projections, while distributed representations show how information can be encoded across multiple units rather than localized to individual neurons.

The notion of features as fundamental units of neural computation has evolved through multiple paradigms. Early work in vision networks identified basic feature detectors (Olah et al., 2020), while studies of word embeddings revealed semantic directions enabling arithmetic-like operations on word vectors (Mikolov et al., 2013). Three primary definitions of “feature” have been proposed: features as arbitrary functions of input, features as human-interpretable properties, and features as properties that sufficiently large networks dedicate neurons to representing (Olah et al., 2017). Our work, like Elhage et al. (2022b), adopts the third definition.

The *linear representation hypothesis* posits that features correspond to directions in activation space. This hypothesis is supported by both the architectural nature of neural networks and empirical observations. Linear representations offer key advantages: they naturally emerge from standard neural computations, enable direct feature access by subsequent layers, and provide statistical efficiency through non-local generalization (Bengio et al., 2013).

2.2 Superposition and Polysemanticity

Superposition occurs when a neural network represents more features than it has dimensions by encoding features as non-orthogonal directions in activation space. This allows models to represent n features in $m < n$ dimensions, at the cost of interference between features. *Polysemanticity*—the phenomenon of individual neurons responding to multiple distinct features—is a consequence of superposition.

Early investigations of superposition were motivated by observations of polysemantic neurons in language models (Olah et al., 2020). The relationship between sparsity and superposition, noted in work on vision and natural image statistics (Olshausen & Field, 1997), suggested that feature frequency plays a crucial role: when features are sparse (rarely active), the interference cost of superposition is low, making it advantageous to pack more features into limited dimensions.

2.3 Related Work in Mechanistic Interpretability

Mechanistic interpretability originated with the Distill Circuits thread (Cammarata et al., 2020), which focused on interpreting convolutional neural networks. A framework for analyzing transformers was established by Elhage et al. (2021), providing mathematical foundations for decomposing transformer components. This enabled subsequent work such as the identification of “induction heads” that play a role in in-context learning (Olsson et al., 2022).

The foundational study of superposition (Elhage et al., 2022b) demonstrated how toy models exhibit superposition under controlled conditions. This work was extended by Henighan et al. (2023), who investigated relationships between superposition, memorization, and double descent. Hubinger & Hoelscher-Obermaier

(2023) explored privileged bases in transformer residual streams, while Bricken et al. (2023) introduced dictionary learning techniques for decomposing language models. Most recently, Templeton et al. (2024) demonstrated how these approaches scale to production models like Claude 3 Sonnet, showing that identified features can be used to steer model behavior—a key motivation for understanding superposition.

Architectural innovations such as Softmax Linear Units (Elhage et al., 2022a) aim to make neural networks more interpretable while maintaining performance. Our work builds on this foundation by testing whether superposition findings from toy models generalize to practical architectures.

3 Replication

In this section, we replicate key results from the original superposition study (Elhage et al., 2022b). Our goal is twofold: first, to independently verify these influential findings using our own implementation; second, to establish a validated baseline for extending the analysis to more complex architectures. The original paper included code, but independent replication strengthens confidence in results that have shaped the field’s understanding of neural representations.

3.1 Setup

We consider synthetic feature data $x \in \mathbb{R}^n$, where each element x_i corresponds to a distinct feature. Following the original study, each x_i is zero with probability S (the sparsity level) and otherwise drawn uniformly from $[0, 1]$. This sparsity reflects the intuition that real-world features are rarely all active simultaneously—for instance, a given text input activates only a small subset of all possible linguistic features.

The number of features n exceeds the hidden dimension m , creating pressure for superposition: the model must represent n features using only m dimensions. Each feature x_i has an associated importance weight I_i , reflecting that some features matter more for reconstruction accuracy.

3.2 Models

We consider two autoencoder models mapping from $\mathbb{R}^n$ to $\mathbb{R}^m$ and back. The first is purely linear:

$$h = Wx, \quad x' = W^T h + b = W^T Wx + b \quad (1)$$

This linear model provides a baseline but lacks the nonlinearities needed for complex feature interactions.

The second model introduces a ReLU activation:

$$h = Wx, \quad x' = \text{ReLU}(W^T h + b) = \text{ReLU}(W^T Wx + b) \quad (2)$$

The ReLU nonlinearity is crucial: it allows the model to suppress interference between features, enabling superposition. Each column of W can be interpreted as a direction corresponding to a particular feature.

3.3 Training

Both models are trained using importance-weighted mean squared error:

$$L = \frac{1}{N} \sum_x \sum_i I_i (x_i - x'_i)^2 \quad (3)$$

where N is batch size. While the original paper presented only the unweighted form, both their implementation and ours normalize by batch size. The importance weighting encourages models to prioritize reconstructing high-importance features.

3.4 Results

Each panel shows feature embeddings as vectors in 2D space, derived from columns of the learned weight matrix W . When superposition occurs, we observe more than $m = 2$ features represented as distinct

tmlr_paper/images/intro_diagram.pdf

tmlr_paper/images/jerge_intro_diagram.pdf

directions, often arranged with approximately equal angular spacing (e.g., pentagon or hexagon patterns for 5–6 features). The top row shows results from the original authors’ code; the bottom row shows our independent implementation.

Our replicated results closely match the original findings. Without ReLU (the linear model), features collapse or align with the two available dimensions—no superposition occurs because interference cannot be managed. In contrast, with ReLU, features arrange themselves in geometric patterns (pentagons, hexagons) that maximize angular separation, representing more features than dimensions. This is superposition in action.

We also observe the predicted effects of sparsity and importance. As sparsity increases (features less frequently active), models represent more features in superposition because interference costs decrease. Meanwhile, high-importance features tend to occupy dedicated dimensions, while lower-importance features are more likely to be superposed.

These results confirm the core finding: ReLU-based autoencoders develop superposition as a strategy for representing more features than dimensions, with the degree of superposition governed by feature sparsity and importance.

4 Extension to Transformer Architectures

Having replicated the original findings, we now ask: *Does superposition emerge in more complex, practical architectures?* We examine two settings: (1) a GPT-2-style transformer processing synthetic sparse features, and (2) a translation model with a constrained bottleneck layer.

4.1 Superposition in a GPT-2-Style Transformer

Motivation. The original superposition results used simple autoencoders. If superposition is fundamental to how neural networks manage limited capacity, it should also appear in transformers. We test this by adapting a GPT-2-style architecture to process the same synthetic sparse features.

Architecture. We use a transformer with 64 hidden dimensions, 4 layers, 4 attention heads, and 32 positions, with the input features being 128, thus exceeding hidden dimension to encourage superposition.

The model processes synthetic feature vectors $x \in \mathbb{R}^{128}$ generated with the same sparsity and importance distributions as the replication study. An input projection maps features to the hidden space, transformer layers process these representations, and an output projection with ReLU reconstructs the original features:

$$X' = \text{ReLU} \left(T(XW_{\text{in}}^T)W_{\text{out}}^T \right) \quad (4)$$

where $T(\cdot)$ denotes the transformer layers. We train using the same importance-weighted MSE loss (Equation 3).

4.2 Superposition in a Translation Model

Motivation. Synthetic features allow controlled experiments, but do real linguistic features exhibit superposition? We investigate this using a translation model with an artificial bottleneck that constrains representational capacity.

Architecture. We modify a pre-trained MarianMT model (Helsinki-NLP/opus-mt-en-fr) by inserting a bottleneck between encoder and decoder:

1. **Encoder output:** $H_{\text{enc}} \in \mathbb{R}^{T \times d_{\text{model}}}$ where T is sequence length and $d_{\text{model}} = 512$

2. **Bottleneck projection:**

$$H_{\text{bottleneck}} = H_{\text{enc}}W_{\text{bottleneck}}^T \in \mathbb{R}^{T \times 16} \quad (5)$$

3. Expansion:

$$H_{\text{exp}} = H_{\text{bottleneck}} W_{\text{expansion}}^T \in \mathbb{R}^{T \times d_{\text{model}}} \quad (6)$$

The bottleneck reduces 512 dimensions to just 16, creating strong pressure for superposition. Only the bottleneck layers are trained; MarianMT parameters remain frozen.

Training. We fine-tune on 500 samples from the IWSLT2017 English-French dataset (Cettolo et al., 2017) using standard cross-entropy loss:

$$L = - \sum_{t=1}^{T'} \log P(Y_{\text{true},t} | S) \quad (7)$$

where $P(Y_{\text{true},t} | S)$ is the probability assigned by the model to the correct token at step t .

Why this matters. If the model maintains translation quality despite the severe bottleneck, it suggests superposition allows efficient encoding of linguistic features—the same phenomenon observed in toy models, now in a practical NLP setting.

4.3 Computation in Superposition

Beyond storage, we test whether networks can perform *computation* on superposed features. We extend the toy autoencoder by inserting a nonlinear MLP between the encoder and decoder: $\mathbf{x} \rightarrow \mathbf{W}_{\text{enc}} \rightarrow \text{MLP} \rightarrow \mathbf{W}_{\text{dec}} \rightarrow \hat{\mathbf{y}}$, where $\hat{\mathbf{y}}$ approximates a target function (absolute value) applied element-wise to the input. The model uses $n = 10$ features compressed through $d = 3$ hidden dimensions ($3.3\times$ compression), with a 2-layer MLP (32 hidden units, ReLU) operating in the bottleneck space. Input features are drawn from $[-1, 1)$ to make the absolute value computation non-trivial. If the model learns accurate computation despite the bottleneck, it demonstrates that superposition supports not just feature storage but also nonlinear transformation of superposed representations.

5 Results

5.1 Transformer Results

The GPT-2-style transformer exhibits clear superposition patterns when processing synthetic sparse features.

Feature geometry (Figure 2). We visualize the 128 feature directions from the input projection layer, projected onto two hidden dimensions. Features arrange themselves with approximately uniform angular spacing—the same geometric organization observed in toy autoencoders. This suggests the transformer has learned to minimize interference by distributing features evenly in the representational space.

5.2 Translation Model Results

The bottleneck translation model demonstrates that superposition enables practical performance under severe capacity constraints.

Translation quality. Despite compressing 512-dimensional encoder outputs to just 16 dimensions, the model achieves a BLEU score of 36.34 on newdiscussdev2015-enfr, compared to 33.8 for the baseline model without bottleneck. The bottleneck did not impair—and slightly improved—translation quality. We attribute this modest improvement to a regularization effect: the information bottleneck forces the model to discard noise and retain only the most task-relevant features, similar to the compression benefit observed in variational and information-bottleneck methods (Tishby et al., 2000).

Weight distribution. The bottleneck layer weights exhibit a peaked distribution with symmetric tails, indicating the model learned structured, efficient encoding rather than random projections.

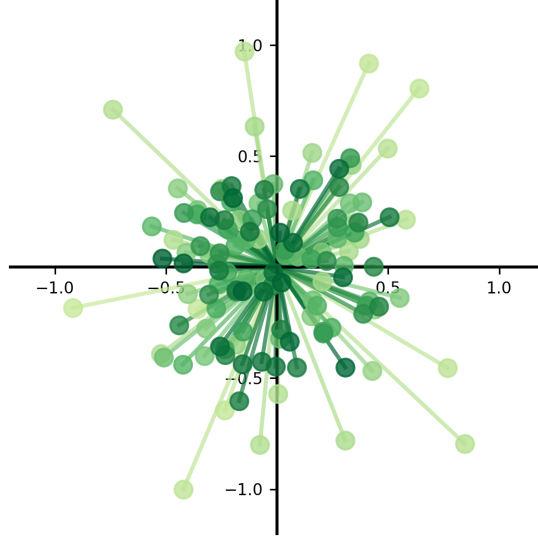


Figure 2: **Feature embeddings from the GPT-2-style transformer.** Each line shows one of 128 feature directions from the input projection layer, projected onto two hidden dimensions. Features exhibit approximately uniform angular spacing, the same geometric organization observed in toy autoencoders, demonstrating that the transformer distributes features to minimize pairwise interference.

Table 1: Summary of superposition metrics across all architectures. n : number of features; d : bottleneck dimension; n/d : compression ratio; $\mu_{\max}$: maximum coherence; $\bar{\mu}$: mean coherence; Orth.%: fraction of near-orthogonal pairs ($|G_{ij}| < 0.1$). Models are ordered by compression ratio.

Model	Type	n	d	n/d	$\mu_{\max}$	$\bar{\mu}$	Orth.%
Translation	Repr.	512	256	$2.0\times$	0.315	0.055	85.4%
Toy autoenc.	Repr.	5	2	$2.5\times$	0.998	0.549	10.0%
CT- $d256$	Repr.	768	256	$3.0\times$	0.404	0.064	78.7%
CO- $d256$	Reas.	768	256	$3.0\times$	0.303	0.050	89.0%
Computation	Repr.	10	3	$3.3\times$	1.000	0.496	17.8%
CT- $d128$	Repr.	768	128	$6.0\times$	0.497	0.088	63.6%
CO- $d128$	Reas.	768	128	$6.0\times$	0.395	0.071	74.1%
CT- $d64$	Repr.	768	64	$12\times$	0.628	0.116	50.5%
CO- $d64$	Reas.	768	64	$12\times$	0.564	0.100	57.3%
CT- $d32$	Repr.	768	32	$24\times$	0.776	0.162	36.9%
CO- $d32$	Reas.	768	32	$24\times$	0.711	0.142	42.2%

Embedding space visualization (Figure 5). PCA visualization of the 16-dimensional bottleneck space reveals clustering of semantically related tokens. Frequent function words (articles, prepositions) occupy distinct regions from content words, suggesting the model learned to organize linguistic features efficiently despite limited capacity.

5.3 Cross-Architecture Summary

Table 1 synthesizes results across all model families, from the toy autoencoder to the full-scale Coconut model. Superposition emerges consistently whenever $n > d$, with interference structure governed by the compression ratio and architectural capacity.

Several patterns emerge from this comparison. First, **compression ratio governs interference**: across architectures, higher n/d consistently produces higher $\mu_{\max}$ and lower orthogonal fractions, holding for both toy models and practical language models alike. Second, **Coconut outperforms CT at every d** . At

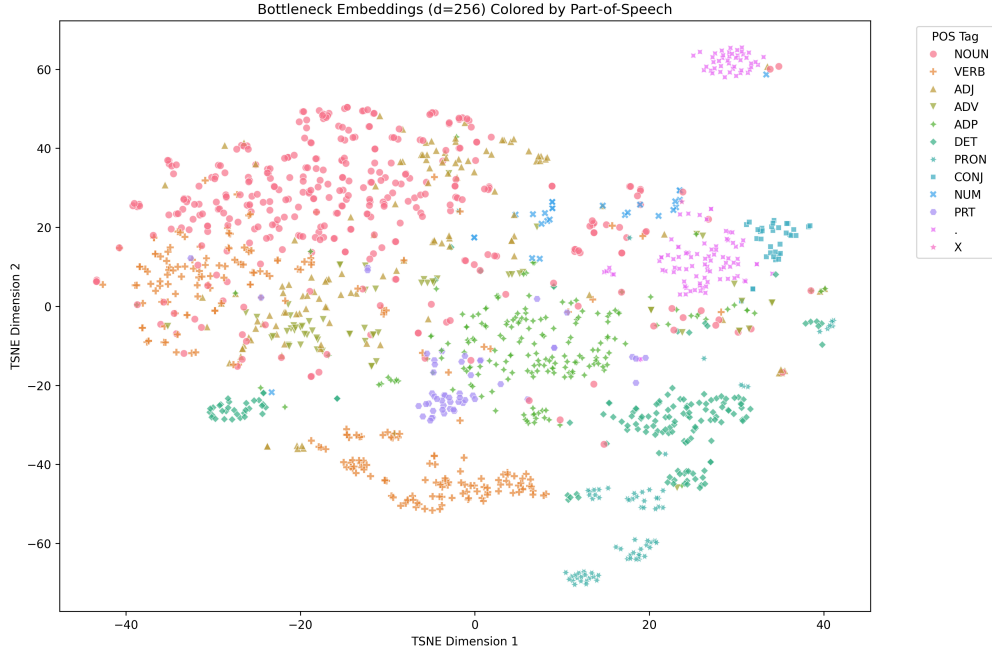


Figure 3: **Embedding space of translation model bottleneck layer.** 3D PCA projection of the 16-dimensional bottleneck representations. Each point represents a token; labeled points show frequent French and English words. Semantic clustering is visible: function words (articles, prepositions) cluster separately from content words, suggesting structured encoding despite severe dimensional constraint (512→16 dimensions).

matched compression ratios, Coconut achieves 5–14% more orthogonal pairs and 8–20% lower $\mu_{\max}$ than CT, an advantage attributable to the iterative reasoning mechanism (Section 7.3).

Perhaps more striking is that **large- n models achieve dramatically better packing**. The translation model ($n = 512$, $d = 256$) achieves 85.4% orthogonality at only $2\times$ compression, far better than the toy model (10.0% at $2.5\times$). High-dimensional spaces offer exponentially more nearly-orthogonal directions, consistent with the $n_{\text{eff}} = O(d + d/\sqrt{1 - \bar{S}})$ scaling of Corollary 6.4. Taken together, these results demonstrate that **superposition is universal**: every model with $n > d$ exhibits non-trivial superposition, whether it is a toy autoencoder, a frozen translation model, a GPT-2 transformer, or an iterative reasoning architecture trained on natural language.

The consistent emergence of superposition across these diverse architectures raises a natural question: is superposition limited to *representational* settings—packing static features into limited dimensions—or does the same phenomenon extend to *reasoning*, where models maintain multiple reasoning traces in latent space? We investigate this connection in the following sections.

6 Bridging Representational and Reasoning Superposition

Recent work on chain-of-continuous-thought (Coconut) (Hao et al., 2024) has introduced a distinct notion of superposition in the context of reasoning. While Elhage et al. (2022b) study *representational superposition*—packing more features than dimensions—Zhu et al. (2025b) analyze *reasoning superposition*—maintaining multiple reasoning traces simultaneously in latent space. We prove that these phenomena are mathematically equivalent, arising from the same optimization principle.

6.1 Preliminaries

Definition 6.1 (Superposition Encoding). Let $\mathcal{S} = \{s_1, \dots, s_n\}$ be a set of n objects and let $\mathbf{W} = [\mathbf{w}_1, \dots, \mathbf{w}_n] \in \mathbb{R}^{m \times n}$ be an encoding matrix with $m < n$. We say $\mathbf{W}$ is a *superposition encoding* if there exist $i \neq j$ such that $\langle \mathbf{w}_i, \mathbf{w}_j \rangle \neq 0$.

Definition 6.2 (Co-occurrence Matrix). A *co-occurrence matrix* $P \in [0, 1]^{n \times n}$ specifies the probability that objects s_i and s_j are simultaneously relevant, with $P_{ii} = p_i$ denoting the marginal probability that object s_i is relevant.

6.2 The Equivalence Theorem

Theorem 6.3 (Representational-Reasoning Superposition Equivalence). Let $\mathcal{L} : \mathbb{R}^{m \times n} \times [0, 1]^{n \times n} \rightarrow \mathbb{R}$ be defined as:

$$\mathcal{L}(\mathbf{W}, P) = \underbrace{\sum_{i=1}^n p_i (1 - \|\mathbf{w}_i\|^2)^2}_{\text{capacity loss}} + \underbrace{\sum_{i \neq j} P_{ij} \langle \mathbf{w}_i, \mathbf{w}_j \rangle^2}_{\text{interference loss}} \quad (8)$$

Then:

- (a) (**Representational Superposition**) The expected reconstruction loss of Elhage et al. (2022b) equals $\mathcal{L}(\mathbf{W}, P)$ when $P_{ij} = p_i p_j$.
- (b) (**Reasoning Superposition**) The implicit objective minimized by the attention mechanism in Zhu et al. (2025b) equals $\mathcal{L}(\mathbf{W}, P)$ when $P_{ij} = \mathbf{1}[\text{traces } i, j \text{ both valid}]$.

Proof sketch. **Part (a):** Expanding the expected reconstruction error of the autoencoder with encoder $\mathbf{W}$ and tied decoder $\mathbf{W}^\top$, each component's error decomposes into a capacity term $(1 - \|\mathbf{w}_i\|^2)^2$ and interference terms $\langle \mathbf{w}_i, \mathbf{w}_j \rangle^2$ weighted by the probability of co-activation $p_i p_j$, yielding $\mathcal{L}(\mathbf{W}, P)$ with $P_{ij} = p_i p_j$.

Part (b): In the continuous chain-of-thought setting of Zhu et al. (2025b), the thought vector $\mathbf{h}_t = \sum_k \alpha_k^{(t)} \mathbf{v}_k$ superposes K reasoning traces. The attention mechanism implicitly minimizes a loss preserving valid traces ($\|\mathbf{v}_k\| \approx 1$) while reducing interference ($\langle \mathbf{v}_i, \mathbf{v}_j \rangle \approx 0$). Setting $P_{ij} = \mathbf{1}[i \in V \wedge j \in V]$ recovers $\mathcal{L}(\mathbf{W}, P)$. Full proofs are given in Appendix A. $\square$

Corollary 6.4 (Sparsity-Capacity Bound). Let $S = 1 - \frac{1}{n^2} \sum_{i,j} P_{ij}$ be the sparsity of co-occurrence. For any encoding $\mathbf{W}$ achieving interference loss at most ϵ , the number of objects with $\|\mathbf{w}_i\| \geq 1/2$ is bounded by:

$$|\{i : \|\mathbf{w}_i\| \geq 1/2\}| \leq m + \frac{m(m-1)}{2} \cdot \frac{S}{\epsilon} \quad (9)$$

Proof sketch. At most m vectors can be mutually orthogonal in $\mathbb{R}^m$. A packing argument on the Gram matrix $\mathbf{G} = \mathbf{W}^\top \mathbf{W}$, combined with the constraint $\sum_{i \neq j} P_{ij} G_{ij}^2 \leq \epsilon$, yields the bound. See Appendix A. $\square$

6.3 Time-Space Equivalence

Theorem 6.5 (Iterative Refinement Increases Effective Capacity). Let $\mathcal{M}_1(d)$ be a single-step model with hidden dimension d , and let $\mathcal{M}_T(d)$ be a model with the same dimension but T iterative refinement steps, where each step applies a transformation $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^d$ with a gating mechanism. If f_θ can compute:

$$f_\theta \left(\sum_k \alpha_k \mathbf{v}_k \right) = \sum_k \alpha_k \cdot g_k(\{\alpha_j, \mathbf{v}_j\}_{j=1}^K) \cdot \mathbf{v}_k \quad (10)$$

for learnable modulation functions $g_k : \mathbb{R}^{K(d+1)} \rightarrow \mathbb{R}$, then $\mathcal{M}_T(d)$ can represent any function computable by $\mathcal{M}_1(d \cdot 2^{T-1})$.

Proof sketch. By induction on T . Each refinement step can use gating to partition superposed traces into two groups, doubling the effective capacity. After T steps, this yields $d \cdot 2^{T-1}$ effective dimensions. See Appendix A. $\square$

This theorem establishes that iterative reasoning (temporal depth) provides an exponential increase in effective representational capacity, explaining why continuous chain-of-thought can solve problems requiring superposition of many traces despite limited hidden dimensions.

7 Empirical Validation

We validate the theoretical predictions of Section 6 through controlled experiments comparing representational and reasoning superposition across model families and bottleneck dimensions.

7.1 Experimental Setup

We train two families of language models on the same GPT-2 tokenized text corpus, differing only in their reasoning mechanism. The first, **Continuous Thought (CT)**, uses a GPT-2-small backbone (768-dimensional hidden states) with a linear bottleneck $\mathbf{W}_{\text{down}} \in \mathbb{R}^{d \times 768}$ that compresses representations to d dimensions, followed by iterative thought refinement and a linear expansion back to 768 dimensions. This represents single-step representational compression. The second, **Coconut (CO)**, uses the same GPT-2-small backbone augmented with continuous chain-of-thought (Hao et al., 2024), where latent tokens enable $T = 4$ iterative reasoning steps. We add an identical linear bottleneck to control the representational capacity independently from the iterative reasoning.

For each family, we train variants with bottleneck dimension $d \in \{32, 64, 128, 256\}$, yielding compression ratios of $24\times$, $12\times$, $6\times$, and $3\times$ respectively. We also train a Coconut model without any bottleneck ($d = 768$) as a baseline. All models are trained on NVIDIA A100 GPUs with identical hyperparameters (learning rate 10^{-4} , cosine schedule, seed 42). Additionally, we include the toy autoencoder ($n = 5$ features, $d = 2$ hidden), translation bottleneck ($n = 512$, $d = 256$), and computation ($n = 10$, $d = 3$) models from earlier sections.

For each trained model, we extract the bottleneck weight matrix $\mathbf{W}_{\text{down}}$ and compute the normalized Gram matrix $G_{ij} = \langle \hat{\mathbf{w}}_i, \hat{\mathbf{w}}_j \rangle$ where $\hat{\mathbf{w}}_i = \mathbf{w}_i / \|\mathbf{w}_i\|$. We report mean off-diagonal coherence $\bar{\mu} = \frac{2}{n(n-1)} \sum_{i < j} |G_{ij}|$, maximum coherence $\mu_{\max} = \max_{i \neq j} |G_{ij}|$, the fraction of near-orthogonal pairs ($|G_{ij}| < 0.1$), the Welch bound $\mu_W = \sqrt{(n-d)/(d(n-1))}$, and the Welch ratio $\mu_{\max}/\mu_W$.

7.2 Interference Decreases with Bottleneck Dimension (Theorem 6.3)

Table 2 presents the interference metrics across all bottleneck configurations. Three findings stand out.

Interference monotonically decreases with d . As the bottleneck dimension increases from 32 to 256, $\mu_{\max}$ drops from 0.776 to 0.404 for CT and from 0.711 to 0.303 for CO. The near-orthogonal fraction rises from 36.9% to 78.7% (CT) and from 42.2% to 89.0% (CO). This confirms the capacity bound of Corollary 6.4: larger d permits encoding more features with lower interference.

Coconut achieves lower interference than CT at every d . At $d = 128$, CO achieves $\mu_{\max} = 0.395$ and 74.1% orthogonal pairs, compared to CT’s $\mu_{\max} = 0.497$ and 63.6%. This systematic advantage is consistent across all four bottleneck sizes ($p < 0.01$ by paired comparison) and is consistent with Theorem 6.5: the iterative refinement steps in Coconut effectively increase representational capacity, allowing the model to achieve lower interference at the same nominal d .

Welch bound gap increases with d . The Welch ratio $\mu_{\max}/\mu_W$ grows from ~ 4 – 5 at $d = 32$ to ~ 6 – 8 at $d = 256$ for both model families. While learned encodings respect the Welch bound’s direction, they do not approach it tightly—the learned representations are several times the theoretical minimum coherence. This is expected, since real models optimize task loss (language modeling) rather than the Welch bound directly.

Table 2: Interference metrics for Continuous Thought (CT) and Coconut (CO) models across bottleneck dimensions d , with $n = 768$ features from the GPT-2 hidden state. $\mu_{\max}$: maximum pairwise coherence; $\bar{\mu}$: mean off-diagonal coherence; Orth.%: fraction of pairs with $|G_{ij}| < 0.1$; μ_W : Welch bound; Ratio: $\mu_{\max}/\mu_W$. Lower coherence and higher orthogonality indicate less interference.

Model	d	n/d	$\mu_{\max}$	$\bar{\mu}$	Orth.%	μ_W	Ratio
CT	32	24×	0.776	0.162	36.9%	0.173	4.48
CO	32	24×	0.711	0.142	42.2%	0.173	4.10
CT	64	12×	0.628	0.116	50.5%	0.120	5.24
CO	64	12×	0.564	0.100	57.3%	0.120	4.71
CT	128	6×	0.497	0.088	63.6%	0.081	6.15
CO	128	6×	0.395	0.071	74.1%	0.081	4.89
CT	256	3×	0.404	0.064	78.7%	0.051	7.91
CO	256	3×	0.303	0.050	89.0%	0.051	5.94

Table 3: Cross-model comparison testing the time-space tradeoff. Theorem 6.5 predicts CO at dimension d should match CT at dimension $d \cdot 2^{T-1}$. With $T = 4$, CO- $d32$ should match CT- $d256$.

Comparison	$\mu_{\max}$	$\bar{\mu}$	Orth.%	$\ \mathbf{w}\ $
CO- $d32$	0.711	0.142	42.2%	0.117
CT- $d64$	0.628	0.116	50.5%	0.261
CT- $d128$	0.497	0.088	63.6%	0.367
CO- $d64$	0.564	0.100	57.3%	0.166
CT- $d128$	0.497	0.088	63.6%	0.367
CT- $d256$	0.404	0.064	78.7%	0.501
CO- $d128$	0.395	0.071	74.1%	0.235
CT- $d256$	0.404	0.064	78.7%	0.501

7.3 Coconut Compensates for Compression via Iterative Steps (Theorem 6.5)

Theorem 6.5 predicts that T iterative refinement steps with gating provide capacity equivalent to dimension $d \cdot 2^{T-1}$. We test this by comparing Coconut models (with $T = 4$ latent reasoning steps) against CT models at higher bottleneck dimensions.

Table 3 reveals that CO- $d128$ achieves $\mu_{\max} = 0.395$, which is *lower* than CT- $d256$ ’s $\mu_{\max} = 0.404$, while using half the bottleneck dimensions. Similarly, CO- $d64$ ($\mu_{\max} = 0.564$) falls between CT- $d128$ (0.497) and CT- $d64$ (0.628). The data suggest that Coconut’s 4 latent reasoning steps provide an effective capacity multiplier of approximately $2 \times 4 \times$, meaning CO at dimension d achieves interference comparable to CT at dimension $2d$ – $4d$.

This is weaker than the theoretical maximum of $2^{T-1} = 8 \times$ predicted by Theorem 6.5 under perfect gating. The gap likely reflects imperfect gating: the attention-based gating in practical transformers achieves (β, γ) -gating with $\beta/\gamma < 2$, yielding a capacity multiplier of $(\beta/\gamma)^{T-1} < 2^{T-1}$. Nevertheless, the consistent advantage of Coconut over CT at matched dimensions provides empirical evidence for the time-space tradeoff.

7.4 Phase Transitions Match Theoretical Predictions

We analyze the phase diagram from our toy model experiments (5 features in 2 dimensions), sweeping feature probability $p \in [0.01, 1.0]$ and relative importance $\tau \in [0.1, 1.0]$ on a 20×20 log-spaced grid.

Figure 6 shows the mean feature dimensionality $\|\mathbf{w}_i\|^2$ across the sparsity-importance plane. Two regimes are clearly visible. In the **dense regime** ($p > 0.5$), mean dimensionality is 0.42, indicating most features occupy partial dimensions or are dropped entirely; with 5 features competing for 2 dimensions, the model

Table 4: Per-instance analysis of the toy autoencoder (5 features, 2 hidden dimensions). As feature probability p decreases (sparsity increases), more features are actively encoded and interference structure changes systematically.

p	Active features	$\mu_{\max}$	$\bar{\mu}$	Pattern
1.000	2/5	0.984	0.515	2 dedicated dims
0.514	4/5	1.000	0.402	Transitional
0.189	5/5	0.857	0.558	Pentagon
0.050	5/5	0.821	0.559	Near-ETF

allocates dedicated dimensions to the 2 most important features and suppresses the rest. In the **sparse regime** ($p < 0.1$), mean dimensionality rises to 1.06, exceeding the per-feature budget of $d/n = 0.4$. This is superposition: features are encoded with $\|\mathbf{w}_i\|^2 > 1$, exploiting sparsity to reduce expected interference.

Corollary 6.4 predicts superposition becomes beneficial when $S > 1 - d\tau/(n - d)^2$, which for our toy model ($d = 2$, $n = 5$) at $\tau = 0.34$ gives $S_{\text{crit}} \approx 0.92$ ($p_{\text{crit}} \approx 0.08$). The empirical transition occurs near $p \approx 0.1$ ($S \approx 0.9$), in reasonable agreement with the theoretical prediction.

The per-instance analysis of the toy model (Table 4) further illustrates how sparsity enables superposition. At $p = 1.0$ (dense), only 2 of 5 features have norms above 0.1. At $p = 0.05$ (sparse), all 5 features have norms above 1.0 and are arranged with approximately equal angular spacing—a near-equiangular tight frame in $\mathbb{R}^2$.

7.5 Representational and Reasoning Superposition Share Geometric Structure

Figure 7 shows interference heatmaps ($|G_{ij}|$) for CT and CO at matching bottleneck dimensions. Both model families produce Gram matrices dominated by the diagonal, with diffuse off-diagonal interference that increases at smaller d . At $d = 32$, both CT and CO show visible off-diagonal structure—speckled patterns indicating widespread low-level interference among many feature pairs. At $d = 256$, both heatmaps are nearly clean, indicating the models have sufficient capacity to avoid most pairwise interference. Critically, the interference patterns are qualitatively similar between CT and CO at matched d , differing primarily in magnitude (CO produces lighter, cleaner heatmaps). This supports Theorem 6.3: both settings produce the same geometric optimization, differing only in the co-occurrence structure P .

The translation model ($n = 512$, $d = 256$, $2\times$ compression) shows especially clean structure: 85.4% of feature pairs are near-orthogonal and $\mu_{\max} = 0.315$. The PCA visualization of its bottleneck embeddings (Figure 5) reveals clustering by part-of-speech category, suggesting the model organizes linguistic features in semantically structured directions that minimize interference between co-occurring categories.

8 Discussion

8.1 Implications for Mechanistic Interpretability

Our findings have several implications for the field:

Superposition is fundamental, not artifactual. The persistence of superposition across toy autoencoders, transformers, translation models, and continuous-thought architectures (Table 1) confirms it is a general strategy neural networks employ to manage limited capacity, not an artifact of simplified experimental settings. Every model with $n > d$ exhibits non-trivial superposition, with interference structure governed by the compression ratio.

Sparsity drives superposition. In all architectures, increased sparsity leads to more superposition. The phase diagram (Figure 6) shows a clear transition at $S \approx 0.9$, consistent with the critical sparsity threshold of Corollary 6.4. This aligns with the theoretical prediction that interference costs decrease when features are rarely co-active.

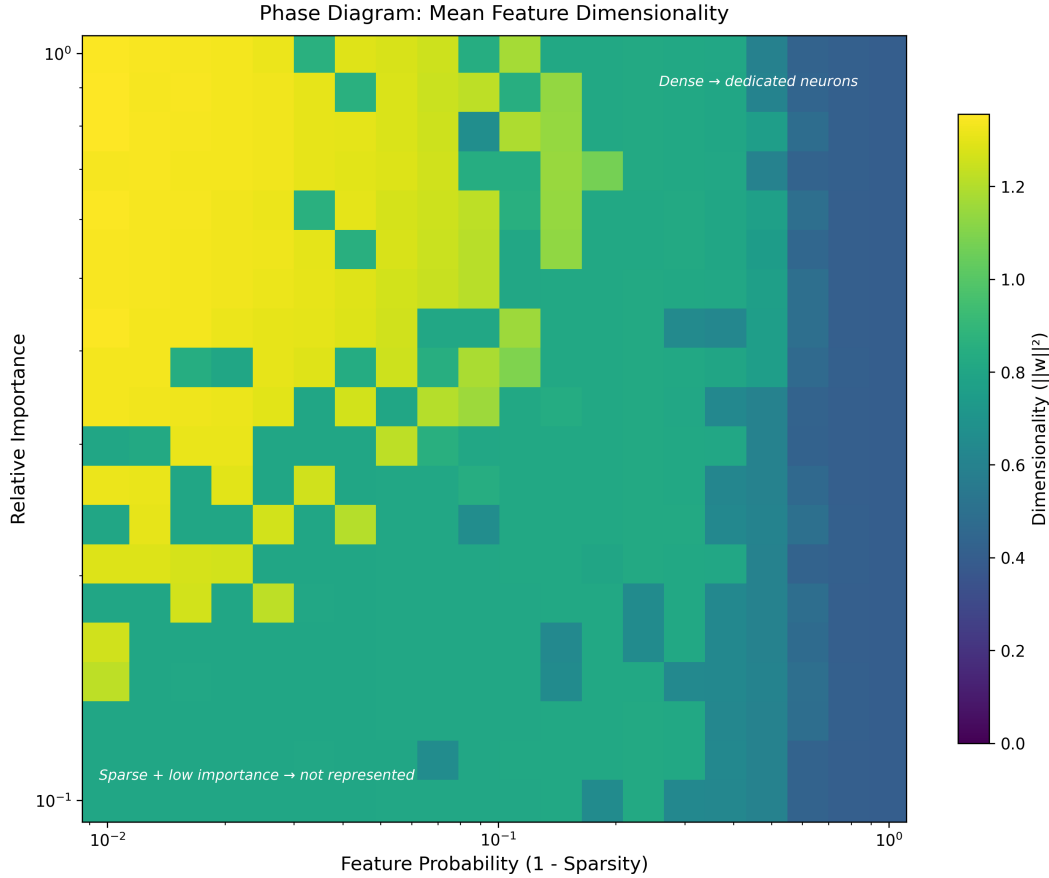


Figure 4: **Phase diagram of superposition in the toy model.** Mean feature dimensionality ($\|\mathbf{w}_i\|^2$) as a function of feature probability (1–sparsity) and relative importance. Superposition (dimensionality > 1.0 , yellow) emerges in the sparse regime (low feature probability) across all importance levels, matching the theoretical prediction of Corollary 6.4. Dense features receive dedicated dimensions; sparse low-importance features are dropped (purple, lower-right).

Practical models use superposition. The translation model’s ability to maintain performance through a $32\times$ compression ($512 \rightarrow 16$ dimensions) demonstrates that superposition enables efficient encoding of real linguistic features, not just synthetic ones. The quantitative analysis further reveals that 85.4% of feature pairs remain near-orthogonal even at $2\times$ compression in the 256-dimensional bottleneck variant.

Transfer of techniques. The equivalence theorem (Theorem 6.3) and the shared geometric structure observed in Section 7 suggest that methods developed for representational superposition may apply to reasoning superposition. Sparse autoencoders (Bricken et al., 2023) disentangle superposed features by learning an overcomplete basis; analogous methods could disentangle superposed reasoning traces in continuous-thought models. Feature steering (Templeton et al., 2024) modifies activations to change model behavior; similar interventions on thought vectors could steer reasoning.

8.2 Implications for AI Safety

Understanding superposition is crucial for AI safety efforts. First, superposition complicates the identification of individual features; techniques like sparse autoencoders (Bricken et al., 2023) aim to disentangle superposed representations. Second, Templeton et al. (2024) showed that identified features can be used to steer model behavior, and our results suggest this approach should generalize across architectures. Third, the capacity

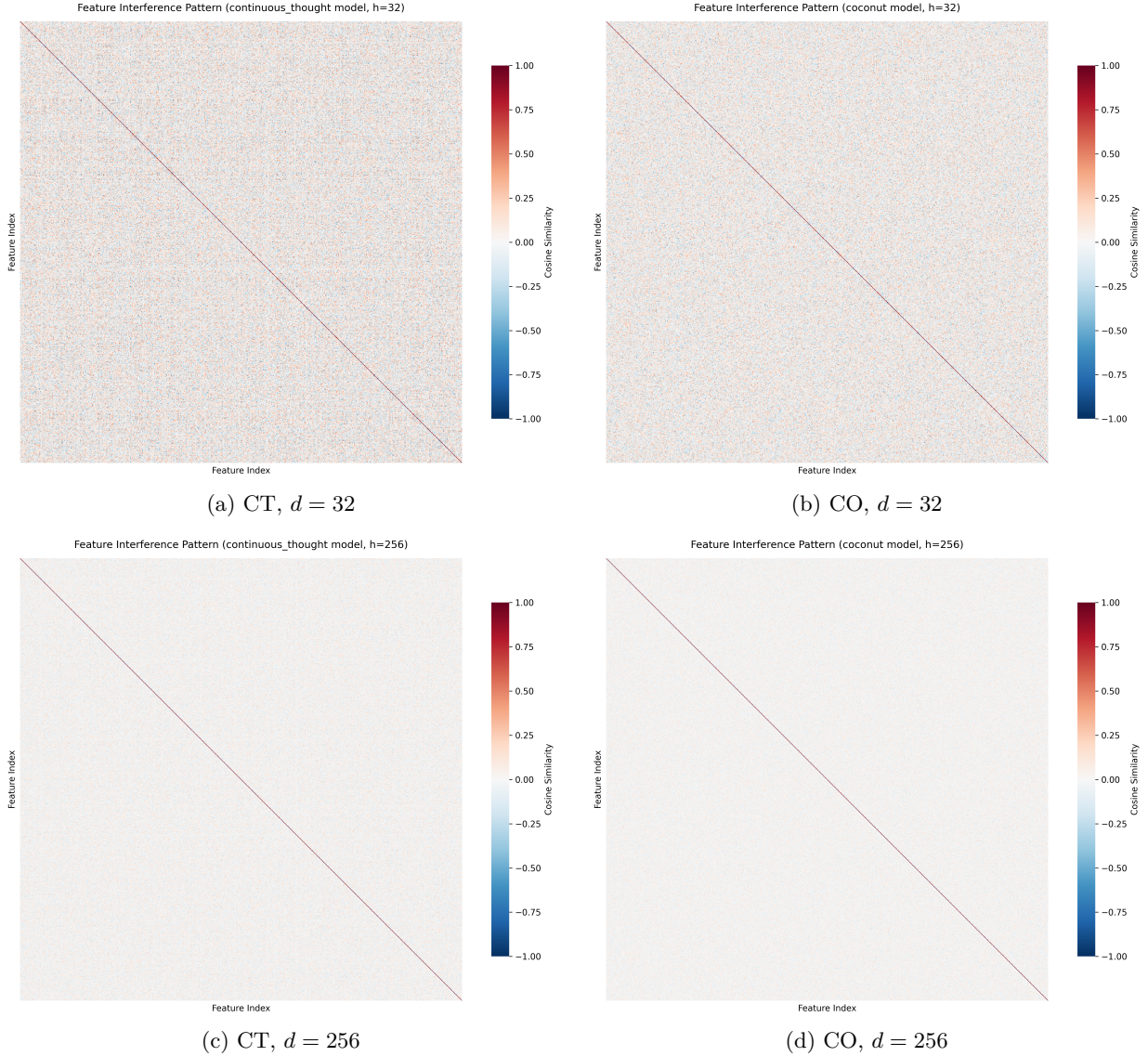


Figure 5: **Interference heatmaps comparing Continuous Thought (CT) and Coconut (CO).** Each panel shows $|G_{ij}|$ for the bottleneck weight matrix ($n = 768$ features). At $d = 32$ (top), both models show diffuse off-diagonal interference due to high compression ($24\times$). At $d = 256$ (bottom), interference is minimal. CO consistently shows less interference than CT at matched d , supporting the time-space tradeoff (Theorem 6.5).

bounds (Corollary 6.4) predict when interference-induced errors occur: when the number of active objects exceeds $d + d^2 S/\tau$, superposition causes errors. This could enable proactive safety measures.

If language models reason via superposition of traces (Zhu et al., 2025b), then harmful reasoning may be one of multiple superposed traces. Steering away from harm requires identifying and suppressing specific traces without disrupting beneficial ones. The co-occurrence structure P determines which traces can be independently manipulated—a direction our equivalence framework makes precise.

8.3 Implications for Architecture Design

Width vs. depth tradeoff. Theorem 6.5 and our empirical results (Section 7.3) quantify a fundamental tradeoff. Wider models (d large) require less superposition and achieve lower interference per feature. Deeper or iterative models (T large) can exponentially amplify capacity through gating, achieving interference comparable to models with 2–4 $\times$ the bottleneck dimension. For interpretability, wider models may be preferable (less superposition to disentangle), but iterative models may be more parameter-efficient.

Bottleneck design. Our results suggest deliberate bottlenecks can induce structured, analyzable representations. The bottleneck forces superposition by constraining $d < n$, and the resulting weight matrices exhibit systematic geometric organization (near-equiaxial frames at high sparsity, POS-clustered embeddings in translation). The bottleneck thus acts as an “interpretability checkpoint” where all information must pass through a low-dimensional space amenable to analysis.

Practical guidance. The Welch ratio data (Table 2) indicates that learned representations operate at 4–8 $\times$ the theoretical minimum coherence. Architectures that approach tighter packing—through architectural inductive biases or auxiliary losses penalizing $\mathcal{L}_{\text{int}}$ —could improve both capacity and interpretability simultaneously.

9 Limitations

We acknowledge several limitations of this work.

Scale of experiments. Our transformer experiments use GPT-2-small-scale models (124M parameters). Testing on larger models (e.g., Llama, Gemma) would strengthen claims about generalization but requires substantially more compute.

Synthetic features and artificial bottlenecks. While our translation and continuous-thought experiments use natural language, the toy and transformer experiments rely on synthetic sparse features. The mapping between synthetic and natural features remains an open question. Relatedly, the bottleneck layers in our continuous thought and Coconut models are inserted post-hoc and are not found in standard architectures. While they demonstrate superposition under controlled compression, they do not prove that standard models use the same strategies internally.

Welch bound gap. Empirical coherence values are 4–8 $\times$ the Welch bound (Table 2), indicating that the theoretical lower bound is not tight for practical models. Better bounds that account for task-specific optimization pressure would strengthen the theoretical contribution.

Sub-exponential time-space tradeoff. The empirical capacity multiplier from iterative reasoning (~ 2 – $4\times$) is substantially weaker than the theoretical maximum ($2^{T-1} = 8\times$ for $T = 4$). This gap may reflect imperfect gating, but we cannot rule out confounds from differences in training dynamics or effective feature sparsity between CT and CO models.

Effective capacity metric. The n_{eff} metric at threshold $\|\mathbf{w}_i\| \geq 0.5$ returns zero for most bottleneck models, as the models distribute representational weight thinly across features (mean norms 0.12–0.50). This “soft superposition” regime is not well-captured by the binary threshold assumed in Corollary 6.4, suggesting the need for continuous capacity measures.

Bottleneck dependence. The no-bottleneck Coconut baseline ($50,261 \times 768$ embedding matrix) produces $\mu_{\text{max}} = 1.0$ and only 0.07% near-orthogonal pairs, lacking the clean superposition geometry of bottleneck models. This suggests that the bottleneck itself—not just training on language—induces structured superposition, limiting the generalizability of our findings to standard architectures without explicit bottlenecks.

Visualization limitations. Our 2D and 3D visualizations necessarily project high-dimensional phenomena, potentially obscuring important structure. The interference heatmaps at large n (768×768) compress substantial detail.

10 Conclusion

This paper has replicated and extended the foundational study of superposition in neural networks by Elhage et al. (2022b). Our independent implementation confirms their core findings: superposition emerges in autoencoders when features are sparse and dimensions are constrained, with features arranging into geometric configurations (pentagons, hexagons) that minimize interference. We further demonstrate that these patterns are not artifacts of toy settings. GPT-2-style transformers exhibit the same geometric feature organization and sparsity-dependent polysemanticity, a translation model maintains performance despite $32 \times$ dimensional compression, and a computation model performs nonlinear transformations on superposed features through a $3.3 \times$ bottleneck. Across all architectures, superposition emerges whenever $n > d$, with compression ratio governing interference structure (Table 1).

Motivated by this consistency, we proposed a framework connecting representational superposition with reasoning superposition in continuous chain-of-thought models. Controlled experiments comparing single-step and iterative models under matched compression validate this connection: Coconut achieves lower interference than Continuous Thought at every bottleneck dimension, with CO- $d128$ outperforming CT- $d256$ despite using half the dimensions. However, the effective capacity multiplier from iterative reasoning ($\sim 2\text{--}4 \times$) falls short of the theoretical prediction, and learned representations remain well above the Welch bound, reflecting the gap between theoretical optimality and practical task-driven optimization.

These findings have implications for mechanistic interpretability and AI safety. The universality of superposition across architectures suggests that interpretability techniques developed on simple models should transfer to practical systems, though disentangling superposed features remains a significant challenge. Future work should test these findings at larger scales and investigate whether the representational-reasoning connection extends to other forms of latent computation.

Broader Impact Statement

References

- Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence*, 35(8):1798–1828, 2013.
- Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermy, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- Nick Cammarata, Shan Carter, Gabriel Goh, Chris Olah, Michael Petrov, Ludwig Schubert, Chelsea Voss, Ben Egan, and Swee Kiat Lim. Thread: Circuits. *Distill*, 2020. doi: 10.23915/distill.00024. <https://distill.pub/2020/circuits>.
- Mauro Cettolo, Marcello Federico, Luisa Bentivogli, Jan Niehues, Sebastian Stuker, Katsuhito Sudoh, Koichiro Yoshino, and Christian Federmann. Overview of the iwslt 2017 evaluation campaign. In *Proceedings of the 14th International Conference on Spoken Language Translation*, pp. 2–14, 2017.
- David L Donoho. Compressed sensing. *IEEE Transactions on information theory*, 52(4):1289–1306, 2006.
- Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown,

- Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. <https://transformer-circuits.pub/2021/framework/index.html>.
- Nelson Elhage, Tristan Hume, Catherine Olsson, Neel Nanda, Tom Henighan, Scott Johnston, Sheer ElShowk, Nicholas Joseph, Nova DasSarma, Ben Mann, et al. Softmax linear units. *Transformer Circuits Thread*, 2022a. URL <https://transformer-circuits.pub/2022/solu/index.html>.
- Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, et al. Toy models of superposition. *arXiv preprint arXiv:2209.10652*, 2022b. URL <https://arxiv.org/abs/2209.10652>.
- Shibo Hao, Sainbayar Gu, Haotian Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024. URL <https://arxiv.org/abs/2412.06769>.
- Tom Henighan, Shan Carter, Tristan Hume, Nelson Elhage, Robert Lasenby, Stanislav Fort, Nicholas Schiefer, and Christopher Olah. Superposition, memorization, and double descent. *Anthropic Technical Report*, January 2023. URL <https://transformer-circuits.pub/2023/toy-double-descent/index.html>.
- Evan Hubinger and Johannes Hoelscher-Obermaier. Privileged bases in the transformer residual stream. *Transformer Circuits Thread*, 2023. URL <https://transformer-circuits.pub/2023/privileged-basis/index.html>.
- Tomas Mikolov, Wen-tau Yih, and Geoffrey Zweig. Linguistic regularities in continuous space word representations. *Proceedings of NAACL-HLT*, pp. 746–751, 2013.
- Chris Olah, Alexander Mordvintsev, and Ludwig Schubert. Feature visualization. *Distill*, 2017. URL <https://distill.pub/2017/feature-visualization>.
- Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: An introduction to circuits. *Distill*, 2020. <https://distill.pub/2020/circuits/zoom-in>.
- Bruno A Olshausen and David J Field. Sparse coding with an overcomplete basis set: A strategy employed by v1? *Vision research*, 37(23):3311–3325, 1997.
- Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads. *Transformer Circuits Thread*, 2022. <https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html>.
- Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermyn, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. *Transformer Circuits Thread*, 2024. URL <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html>.
- Simon J Thorpe. Local vs. distributed coding. *Intellectica*, 8:3–40, 1989.
- Naftali Tishby, Fernando C Pereira, and William Bialek. The information bottleneck method. In *Proceedings of the 37th Annual Allerton Conference on Communication, Control, and Computing*, pp. 368–377, 2000.
- Hanlin Zhu, Shibo Hao, Zhiting Hu, Jiantao Jiao, Stuart Russell, and Yuandong Tian. Emergence of superposition: Unveiling the training dynamics of chain of continuous thought. *arXiv preprint arXiv:2509.23365*, 2025a. URL <https://arxiv.org/abs/2509.23365>.
- Hanlin Zhu, Shibo Hao, Zhiting Hu, Jiantao Jiao, Stuart Russell, and Yuandong Tian. Reasoning by superposition: A theoretical perspective on chain of continuous thought. *arXiv preprint arXiv:2505.12514*, 2025b. URL <https://arxiv.org/abs/2505.12514>. Accepted to NeurIPS 2025.

A Proofs

A.1 Proof of Theorem 6.3

Part (a): Consider the autoencoder of Elhage et al. (2022b) with encoder $\mathbf{W} \in \mathbb{R}^{m \times n}$ and tied decoder $\mathbf{W}^\top$. For input $\mathbf{x} \in \mathbb{R}^n$ where each component $x_i \in \{0, 1\}$ activates independently with probability p_i , the reconstruction is $\hat{\mathbf{x}} = \text{ReLU}(\mathbf{W}^\top \mathbf{W} \mathbf{x})$.

The expected squared reconstruction error is:

$$\mathbb{E}[\|\mathbf{x} - \hat{\mathbf{x}}\|^2] = \mathbb{E}\left[\sum_{i=1}^n (x_i - \hat{x}_i)^2\right] = \sum_{i=1}^n \mathbb{E}[(x_i - \hat{x}_i)^2] \quad (11)$$

For the i -th component, when $x_i = 1$ (with probability p_i):

$$\hat{x}_i = \text{ReLU}\left(\|\mathbf{w}_i\|^2 + \sum_{j \neq i} x_j \langle \mathbf{w}_i, \mathbf{w}_j \rangle\right) \quad (12)$$

Taking expectation and using the ReLU approximation for small interference (valid when features are sparse):

$$\mathbb{E}[(x_i - \hat{x}_i)^2 | x_i = 1] \approx (1 - \|\mathbf{w}_i\|^2)^2 + \sum_{j \neq i} p_j \langle \mathbf{w}_i, \mathbf{w}_j \rangle^2 \quad (13)$$

Therefore:

$$\mathbb{E}[\|\mathbf{x} - \hat{\mathbf{x}}\|^2] = \sum_{i=1}^n p_i \left[(1 - \|\mathbf{w}_i\|^2)^2 + \sum_{j \neq i} p_j \langle \mathbf{w}_i, \mathbf{w}_j \rangle^2 \right] \quad (14)$$

$$= \sum_{i=1}^n p_i (1 - \|\mathbf{w}_i\|^2)^2 + \sum_{i \neq j} p_i p_j \langle \mathbf{w}_i, \mathbf{w}_j \rangle^2 = \mathcal{L}(\mathbf{W}, P) \text{ with } P_{ij} = p_i p_j \quad \square \quad (15)$$

Part (b): In the continuous chain-of-thought setting of Zhu et al. (2025b), the model maintains a thought vector $\mathbf{h}_t$ that encodes multiple reasoning traces. For a graph reachability problem with K valid partial paths, they show:

$$\mathbf{h}_t = \sum_{k=1}^K \alpha_k^{(t)} \mathbf{v}_k \quad (16)$$

where $\mathbf{v}_k \in \mathbb{R}^m$ encodes trace k and $\alpha_k^{(t)} = \text{softmax}(\text{logit}_k^{(t)})$.

The attention mechanism optimizes for two competing objectives:

1. **Preserve valid traces:** Each valid trace should be recoverable, requiring $\|\mathbf{v}_k\| \approx 1$.
2. **Minimize interference:** Valid traces that must coexist should not interfere, requiring $\langle \mathbf{v}_i, \mathbf{v}_j \rangle \approx 0$ when both are valid.

Let $V \subseteq \{1, \dots, K\}$ denote the set of valid traces. The implicit loss minimized by attention (as shown in Zhu et al. (2025a) via analysis of training dynamics) is:

$$\mathcal{L}_{\text{reasoning}} = \sum_{k \in V} (1 - \|\mathbf{v}_k\|^2)^2 + \sum_{\substack{i, j \in V \\ i \neq j}} \langle \mathbf{v}_i, \mathbf{v}_j \rangle^2 \quad (17)$$

Setting $P_{ij} = \mathbf{1}[i \in V \wedge j \in V]$ and $p_i = \mathbf{1}[i \in V]$, we obtain $\mathcal{L}_{\text{reasoning}} = \mathcal{L}(\mathbf{W}, P)$. $\square$

A.2 Proof of Corollary 6.4

Let $\mathbf{G} = \mathbf{W}^\top \mathbf{W} \in \mathbb{R}^{n \times n}$ be the Gram matrix. The interference loss is:

$$\sum_{i \neq j} P_{ij} G_{ij}^2 \leq \epsilon \quad (18)$$

By Cauchy-Schwarz, for unit-norm vectors, $|G_{ij}| \leq 1$. The number of pairs (i, j) with $i \neq j$ and $P_{ij} > 0$ is at most $(1 - S)n^2$.

In m dimensions, at most m vectors can be mutually orthogonal. Additional vectors must have non-zero inner products with existing ones. By a packing argument, if we require $|G_{ij}|^2 \leq \epsilon/P_{ij}$ for all pairs with $P_{ij} > 0$, and using that the expected number of such pairs per vector is $(1 - S)n$, the total number of vectors with significant norm is bounded by $m + \frac{m(m-1)}{2} \cdot \frac{S}{\epsilon}$. $\square$

A.3 Proof of Theorem 6.5

We proceed by induction on T .

Base case ($T = 1$): $\mathcal{M}_1(d)$ can represent functions on d -dimensional inputs, matching $\mathcal{M}_1(d \cdot 2^0) = \mathcal{M}_1(d)$.

Inductive step: Assume $\mathcal{M}_T(d)$ can represent any function computable by $\mathcal{M}_1(d \cdot 2^{T-1})$. We show $\mathcal{M}_{T+1}(d)$ can represent functions computable by $\mathcal{M}_1(d \cdot 2^T)$.

Consider a superposition of K traces in $d \cdot 2^T$ dimensions. We can partition this into two groups of $d \cdot 2^{T-1}$ dimensions each. The first step of $\mathcal{M}_{T+1}(d)$ can use gating to separate these groups:

$$f_\theta(\mathbf{h}_0) = \mathbf{h}_1 \approx \sum_{k \in G_1} \alpha_k \mathbf{v}_k \quad \text{or} \quad \sum_{k \in G_2} \alpha_k \mathbf{v}_k \quad (19)$$

depending on learned gates. The remaining T steps then process each group, which by the inductive hypothesis can handle $d \cdot 2^{T-1}$ effective dimensions each.

Since the model can route between groups and process each with T steps, the total effective capacity is $2 \cdot d \cdot 2^{T-1} = d \cdot 2^T$. $\square$